

Stacked Isolation Forest + XGBoost

Training report - new_isolationforest_xgboost_hybrid/
CICIDS2018 cleaned dataset, 15 attack classes kept separate

1. What this model is

This is a different design from the project's earlier hybrid (isolation_forest_and_xgboost_model/), which GATES: the Isolation Forest's anomaly score is compared to a threshold, and only the flows it flags reach XGBoost, using their raw features only (the score itself is discarded after the threshold check). This model STACKS instead: the Isolation Forest's anomaly score is computed for every single flow and added as one more input column. A single XGBoost classifier then makes the only decision, for every flow, across all 15 original classes. There is no separate stage-1 verdict or stage-1 table any more - Isolation Forest only hands over a number; XGBoost decides.

Unlike an earlier attempt in this project (retraining_hybrid/, since deleted by the user), the 15 original classes are kept fully separate here. SSH-Bruteforce, FTP-BruteForce, Brute Force-Web and Brute Force-XSS are NOT merged into one "Brute Force" class, and the DoS / DDoS sub-types are not merged either.

2. Data and feature selection

Source: original_csv/cleaned_dataset_2.csv, a copy of the full cleaned CICIDS2018 dataset (11,872,582 rows) with 15 columns removed, leaving 63 features + Label. The 79-column dataset itself is unchanged; this is a separate derived copy made for this model.

Columns removed, and why:

Reason	Columns
All-zero in the whole cleaned dataset (8)	Bwd PSH Flags, Bwd URG Flags, Fwd Byts/b Avg, Fwd Pkts/b Avg, Fwd Blk Rate Avg, Bwd Byts/b Avg, Bwd Pkts/b Avg
Exact copy of another column (7, confirmed)	Subflow Fwd Pkts = TotLen Fwd Pkts; Subflow Fwd Byts = TotLen Fwd Pkts; Subflow Bwd Pkts = Tot Bwd Pkts

Sample: every attack row plus 1,000,000 random Benign rows (same recipe as every other model in this project).
Split: 80% train / 20% test, stratified by class, seed 42. Train rows: 1,863,347 | Test rows: 465,837.

3. Stage 1: Isolation Forest (anomaly score, not a gate)

Setting	Value
Trees	300
Rows per tree (max_samples)	8192
Contamination	0.02
Fitted on	Benign rows of the training split only
Transform	log1p(x + 1) then StandardScaler
Informational ROC-AUC	0.895
Informational threshold (not used to gate)	0.4542

300 trees and 8192 rows/tree were chosen after measuring a real improvement over an earlier 200/4096 setting: ROC-AUC rose from 0.874 to 0.896 and balanced accuracy from 83.8% to 86.3%, mostly from DoS recall rising from 59% to 92%. The threshold above is reported only to show how separable normal and attack traffic are on the score alone - it plays no part in the pipeline.

4. Stacking step and Stage 2: XGBoost (the final classifier)

Every row's Isolation Forest score is attached as column 64 (named IF_Anomaly_Score), after the 63 real features. XGBoost is trained on all 64 columns, with square-root "balanced" sample weights so rare classes (as few as

31-39 training rows) are not ignored. Settings: 200 trees, depth 10, learning rate 0.1, subsample 0.8, colsample_bytree 0.8, objective multi:softprob, 15 classes.

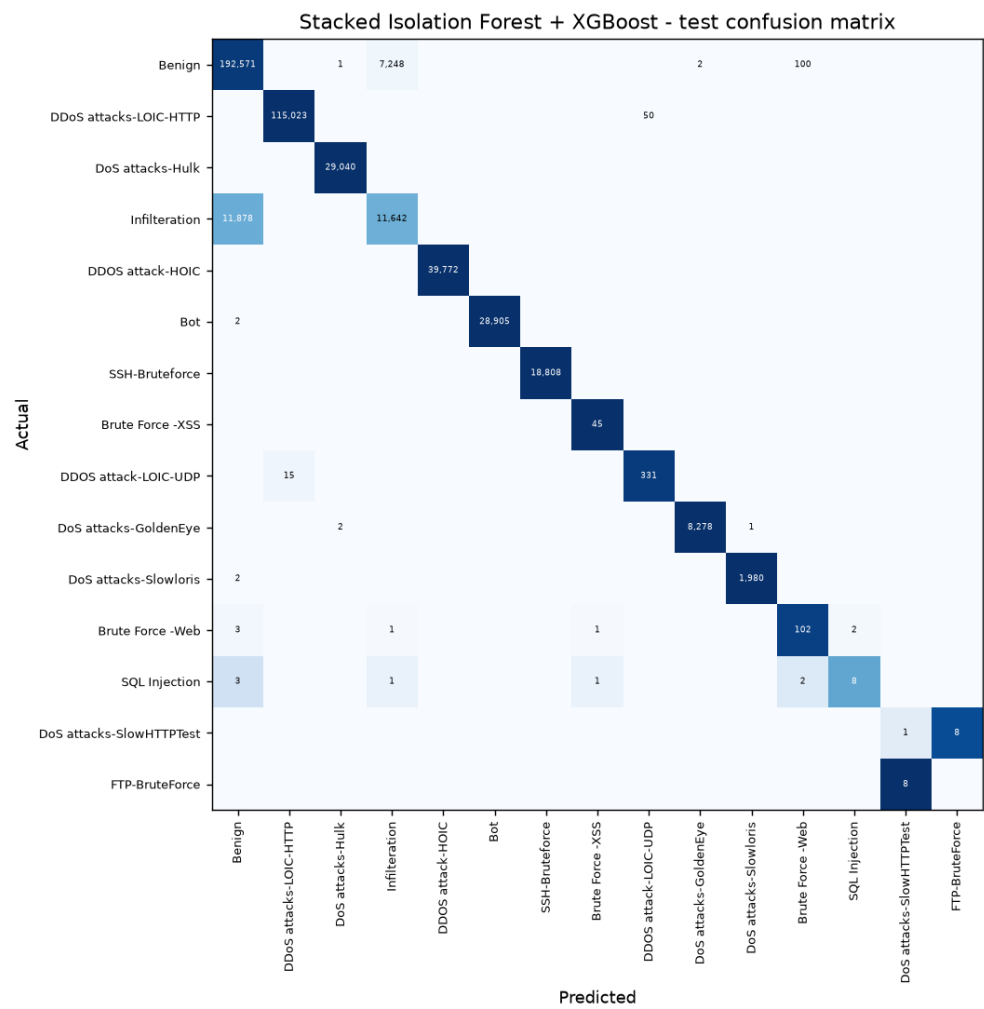
5. Final test result (465,837 held-out rows, unseen by both models)

Overall accuracy: 95.85% Classes at or above 95% recall: **10 of 15**

Class	Test rows	Precision	Recall	F1
Benign	199,922	94.2%	96.3%	0.952
DDoS attacks-LOIC-HTTP	115,073	100.0%	100.0%	1.000
DDoS attack-HOIC	39,772	100.0%	100.0%	1.000
DoS attacks-Hulk	29,040	100.0%	100.0%	1.000
Bot	28,907	100.0%	100.0%	1.000
Infiltration	23,520	61.6%	49.5% < 95%	0.549
SSH-Bruteforce	18,808	100.0%	100.0%	1.000
DoS attacks-GoldenEye	8,281	100.0%	100.0%	1.000
DoS attacks-Slowloris	1,982	99.9%	99.9%	0.999
DDoS attack-LOIC-UDP	346	86.9%	95.7%	0.911
Brute Force -Web	109	50.0%	93.6% < 95%	0.652
Brute Force -XSS	45	95.7%	100.0%	0.978
SQL Injection	15	80.0%	53.3% < 95%	0.640
DoS attacks-SlowHTTPTest	9	11.1%	11.1% < 95%	0.111
FTP-BruteForce	8	0.0%	0.0% < 95%	0.000

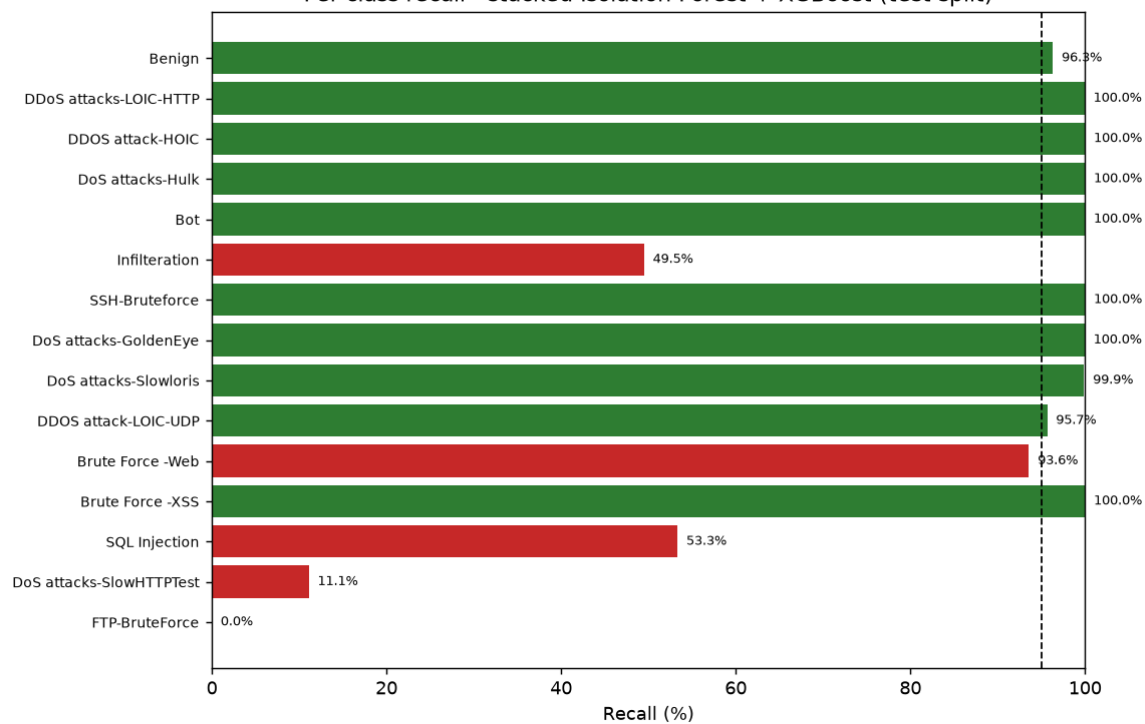
Infiltration (49.5% recall, 23,520 test rows) is the one large class still below target - the same class that is weak in every model in this project, because its flows are deliberately built to look like normal traffic. Brute Force-Web, SQL Injection, SlowHTTPTest and FTP-BruteForce are below target too, but each has under 110 test rows (as few as 8), so those numbers are not statistically reliable - the same limitation the 1_xgboost_model/ report notes for these classes.

6. Confusion matrix (test split)



7. Per-class recall

Per-class recall - stacked Isolation Forest + XGBoost (test split)



8. Live traffic tests

kali_patator_1s.csv (Java CICFlowMeter-v3, Kali-to-Kali Patator with persistent connections, 182 confirmed attack flows): $171/182 = 93.96\%$ correctly named SSH-Bruteforce. This matches the result the earlier (gated) models gave on the same file, showing that stacking the anomaly score and keeping the 15 classes separate did not regress this result.

anivrath1.csv (live capture from samiksha_capture/, 1,776 flows, a real SSH brute force with password Zebra123, ground truth confirmed by the user): the Isolation Forest's score was above its informational threshold for every single flow (minimum 0.5686 vs threshold 0.4542 - the whole capture looks anomalous on that signal alone). Despite that, XGBoost still labelled 100% of the flows Benign. This shows that adding the anomaly score as a feature does not by itself fix the known flow-shape mismatch: this capture's flows (median about 4 packets each way, about 5 ms) are far from the SSH-Bruteforce pattern learned from training (about 22 packets, about 375 ms), and the score alone did not give XGBoost enough new information to bridge that gap.

9. Conclusion

Stacking the Isolation Forest's score into XGBoost, with all 15 classes kept separate, reaches about the same offline accuracy as plain XGBoost alone (1_xgboost_model/: 95.70%), while keeping SSH-Bruteforce at a full 100% recall. Keeping the classes separate, rather than merging brute-force types together, mattered in practice: the earlier merged retrain only got 8.79% right on kali_patator_1s.csv, against 93.96% here. What stacking has NOT been shown to fix is the Infiltration weakness or the live flow-shape mismatch - both are still open problems carried over from every earlier model in this project.